

AI Infra · 训练确定性与推理系统

把浮点顺序、路由决策与跨 step 状态机放进同一条验证链

- Deterministic
- Attention
- Router Replay
- 状态机

岗位能力画像

岗位画像：既考 Attention / deterministic backward 的底层机制，也考推理状态在动态请求下的正确性。

高频深挖：题目背后的判断

确定性为什么会慢？

全局顺序票据让本可独立的更新链排队；先用 trace 判断是不是依赖，而不是算力不足。

固定每条归约顺序，扩大独立链并发。

新实现如何验收？

区分与参考实现的误差边界和新实现自身多次运行的可复现性。

确定性目标是可重复，不等于复制旧累加顺序。

路由回放解决什么？

只锁定关键离散决策，避免把算子/数值路径差异放大成 top-k 分叉。

回放减少一类路径漂移，不等于证明某条路由更优。

跨 step state 最容易漏什么？

mixed batch、reorder、slot reuse、恢复请求都在考状态机，而非单次 kernel 输出。

生命周期语义必须先于吞吐优化。

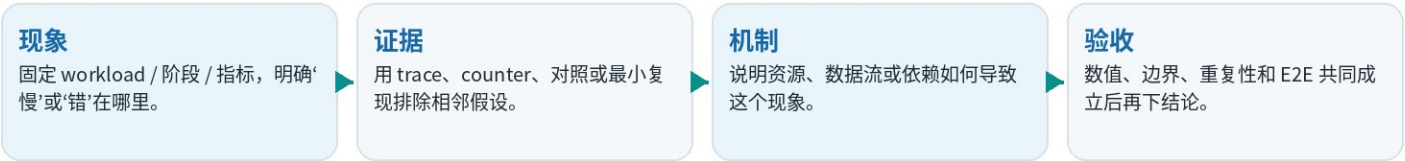
项目叙事的通用深挖链



答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实



一题答透：确定性 backward：让顺序可控，让并行不闲置

问题 “局部注意力的 deterministic backward 比 forward 慢，如何定位和改造？”

1. 锁热点

用时间线和 kernel 指标确认慢点在特定梯度更新链，而不是泛泛的 Attention。

2. 解释等待

观察大量 cluster 等全局 ticket 且 SM/DRAM 未饱和，说明关键问题是依赖串行化。

3. 重排依赖

缩小票据到实际窗口，按反向访问方向消费热数据；每条归约链仍保持固定顺序。

4. 验证边界

分别验数值参考、新路径自身重复性、极端长度 fallback 与端到端回归。

把推理链画出来



常见误区

× 把误差可接受和 bitwise deterministic 混为一谈。

× 只改调度，不解释每条归约链的顺序是否仍固定。

× 只覆盖单请求，遗漏恢复、reorder 和 slot reuse。

术语速懂 | 面试怎么答

术语速懂

Router Replay

回放 token 在关键层的离散 top-k 决策，避免微小数值差异在边界放大成路径分叉。

Deterministic backward

固定梯度归约链的消费顺序，同时让无依赖窗口并发，避免全局票据串行。

面试中怎么答

- ① 先解释离散 top-k 和浮点归约为何会放大微小扰动。
- ② 再明确记录对象、对齐方式和真实依赖范围，而非全局串行。
- ③ 分别看 reference 误差、重复性、动态状态和端到端回归。

白板 / 手撕：白板题：row-wise max 与 online softmax

题面 给 [M,N] 矩阵按行求 max；再说明确分块 softmax 怎样维护 running max、normalizer 和 accumulator。

C++ / CUDA-style · online softmax merge

核心实现

```
01 struct Online { float m = -INFINITY, l = 0.f, acc = 0.f; };
02 Online merge(Online a, Online b) {
03     if (b.l == 0.f) return a;    // all-masked block
04     if (a.l == 0.f) return b;
05     float mn = fmaxf(a.m, b.m);
06     float wa = expf(a.m - mn);
07     float wb = expf(b.m - mn);
08     return {mn, wa * a.l + wb * b.l,
09             wa * a.acc + wb * b.acc};
10 }
11 // final scalar output: acc / l
```

写代码时怎么讲

1. row max：thread-local max → warp shuffle → 必要时 shared/block reduce → thread 0 store。
2. N 非整除时 mask；明确 NaN 和 -inf 语义。
3. online softmax：合并两个块时先更新 m，再按缩放因子合并 l 与 acc。
4. 解释为什么无需 materialize 完整 score / probability 矩阵。

实现重点

- 实际 attention 中 acc 是每个 value 维度的向量累积。
- 全 mask 与 -inf 初始化需单独处理，避免 NaN。
- 块内先算局部状态，块间再用此式合并。

边界、测试与追问

最少要说明的验证

- 实际 attention 中 acc 是每个 value 维度的向量累积。
- 全 mask 与 -inf 初始化需单独处理，避免 NaN。
- 块内先算局部状态，块间再用此式合并。

面试继续追问

- top-k 边界为什么会放大微小误差？
- 不切 K 怎样帮助固定归约顺序？
- 状态机如何做请求恢复的回归测试？